

Deep learning alchemy

CS312

Teaching staff



Suhas Kotha



Tatsunori Hashimoto



Kaiyue Wen



Tristan Thrush

(Note: this course is recorded by CGOE and recordings will be uploaded publicly)

Deep Learning Everywhere

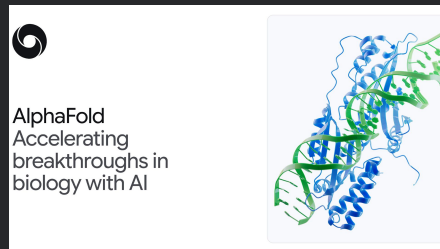
Many domains (and settings), but deep learning in all of them



LLMs



Waymo



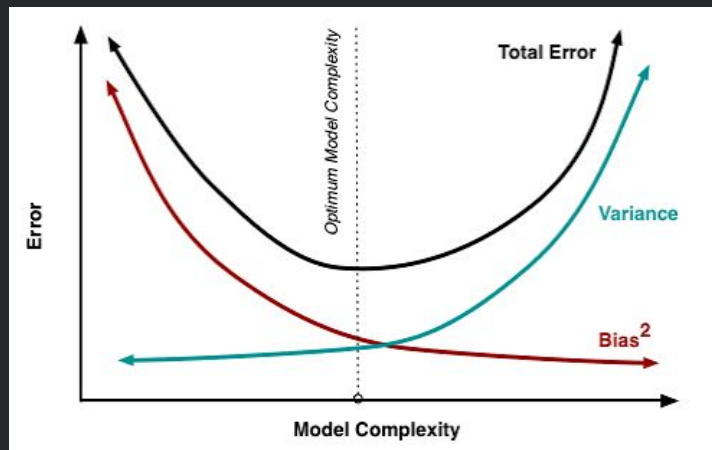
AlphaFold



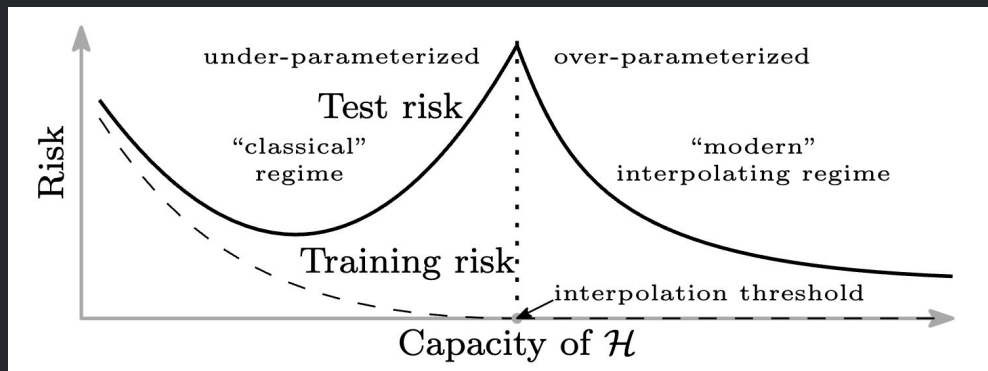
AlphaGo

On the unreasonable effectiveness of deep learning

Pre-deep learning intuition..



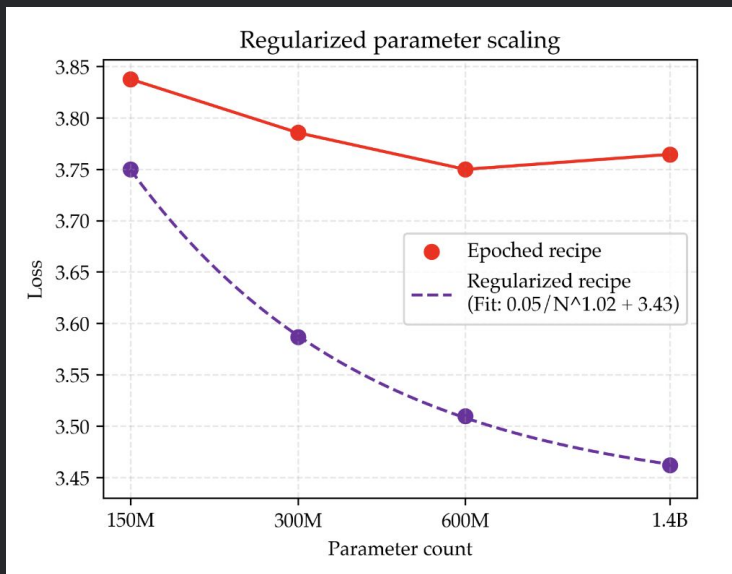
Post-deep learning phenomena



We need new intuitions for deep learning

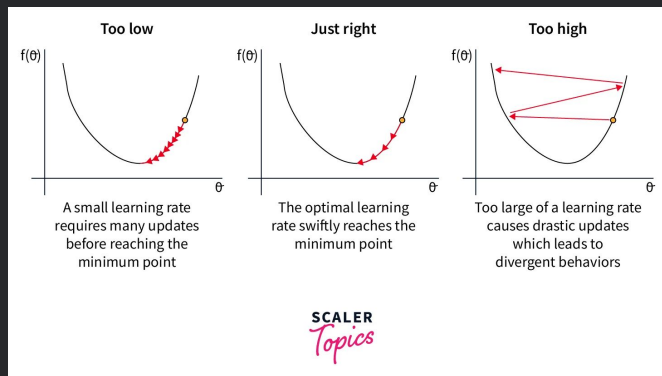
... and these phenomena are brittle!

Even this might be false if we properly tune the hyperparameters. Our theories of deep learning still requires empirical know-how, and we are still unsure when our cute mental models lead us astray

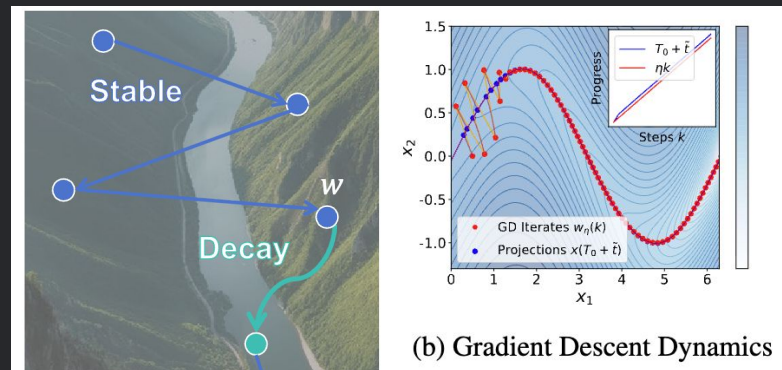


Other examples of “new” intuitions..

“Ideal” convex intuition



Deep learning training reality



Highly chaotic training

Alchemy



GLU Variants Improve Transformer

Noam Shazeer
Google
noam@google.com

February 14, 2020

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

No general theory of deep learning (yet). Intuition and trial-and-error dominate.

Teaching (and research) often does not reflect this reality

Embracing the alchemy

What is the right way to teach (and understand) deep learning in 2026?

We think there are “wrong” ways

- Deep learning with “classical” theory (convex opt, uniform generalization)
- Reading and memorizing phenomena (these are often brittle!)
- Project based learning (the only one doing learning is claude)

Our view

Reliably predicting the outcome of experiments leads to understanding

This class itself is an experiment! Based on two ideas..

- Understanding requires interaction and mastery
- Prediction forces you to experiment and build your own 'world model'

Example prediction problem

From the website (Difficulty Hard): We train the default 8 layer transformer as well as a variant where instead of having the residual connection come from the previous layer, we take 0.5 contribution from the previous layer and 0.5 contribution from two layers before (except for the first layer). For each method, we take the best loss over six learning rates $\{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$. What is your prediction of the loss difference between each method ($L_{\text{mixed residual}} - L_{\text{default}}$)?

▼ Core diff

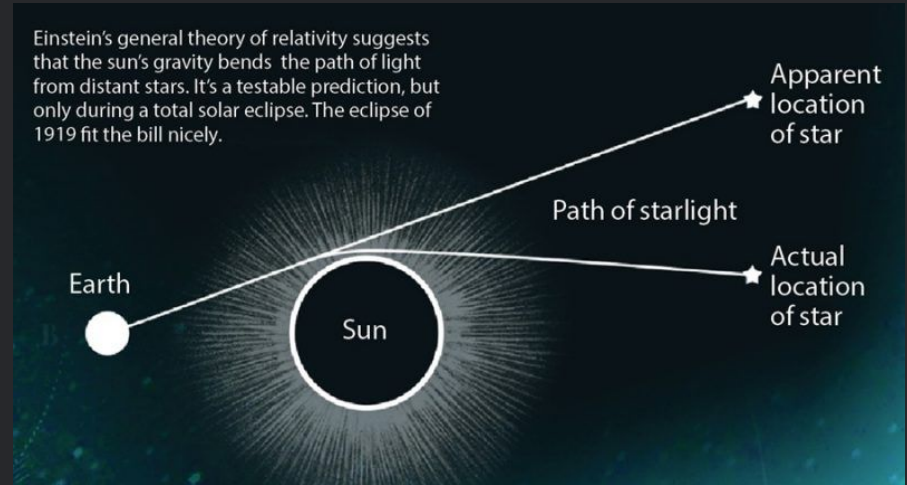
```
@@ -34,3 +34,9 @@ Transformer.forward
x_two_back = None
for i, block in enumerate(self.blocks):
-   x = block(x)
+   update = block.update(x)
+   residual = (
+       x if i == 0
+       else 0.5 * x + 0.5 * x_two_back
+   )
+   x_two_back, x = x, residual + update
```

Why prediction?

Prediction is central to science!



Early astronomy based on predictions of eclipses and other phenomena



Eddington's experiments on relativity

Why prediction?

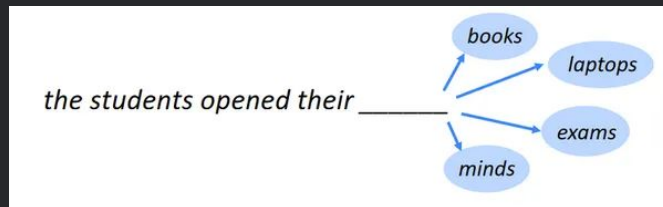
Prediction is central to machine learning!

ImageNet Leaderboard

2012 2019 @PapersWithCode

Date: 2019-07-23

Rank	Top 1 Accuracy	Model	Date
#1	86.4%	FixResNeXt-101 32x48d	2019-06-14
#2	85.4%	ResNeXt-101 32x48d	2018-05-02
#3	85.1%	ResNeXt-101 32x32d	2018-05-02
#4	84.4%	EfficientNet	2019-05-28
#5	84.3%	GPIPE	2018-11-16
#6	84.2%	ResNeXt-101 32x16d	2018-05-02
#7	83.9%	AmoebaNet-A	2018-02-05
#8	83.7%	FixPNASNet-5	2019-06-14
#9	83.6%	MultiGrain PNASNet	2019-02-14
#10	83%	MultiGrain SENet154	2019-02-14



Much like language modeling, you'll be asked to (pre) train however you want (in assignments)
And then, you'll be tested on your ability to predict the behavior of a complex system

What do we predict?

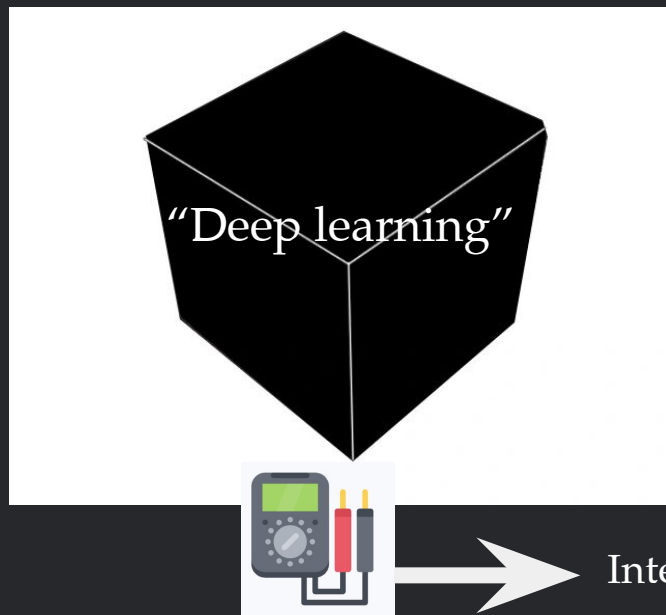
We want you to have a 'world model' for deep learning

Optimizer

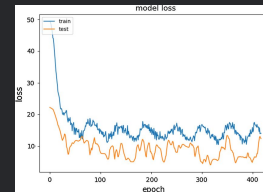
Architecture

Data

Loss (and more)



Performance (loss)



Internal metrics (activations, norms)

Logistics (Assignments)

Core topics: 7 Assignments

1. Basic deep learning skills (Hyperparams, debugging, etc)
2. Optimization 1 (Scaling, invariants, hyperparameters)
3. Optimization 2 (Sharpness, chaos)
4. Architecture 1 (Representation, model capacity)
5. Architecture 2 (Stability and signal propagation)
6. Generalization (Data efficiency)
7. Exotic (Chef's choice)

Assignments are most of the work, but they count for (basically) zero percent.

We are calibrated for 5 units, so ≥ 15 hrs per week.

Logistics (Assignments)

Why are assignments 0% of the grade?

- Many of you will claude code it. At best this is useless, at worst unfair.
- This lets us have a permissive AI policy (do whatever you want)

Why do the assignments if it's 0% of the grade?

- They are quiz prep (and quizzes are 85% of the grade)
- You cannot get quiz partial credit without citing your assignments
- Tutorials will test your understanding in person

Logistics (Assignment - bonus)

The one exception is the “bonus question”

Optional exploration

To explore the topics in this assignment more broadly, construct your own prediction problem. This final exercise is not related to a quiz question, but can lead to bonus points on the quiz.

Problem 9: Your own prediction problem

You have now explored the phenomena in this assignment thoroughly, and may have found unexpected or surprising phenomena. We would like to help make better, more interesting quizzes based on your insights.

Include the following in your exploration:

- (a) A question writeup, similar to the sample questions in this assignment. Answers should be objectively checkable based on the execution of a piece of code (ideally, multiple choice or numerical prediction).
- (b) A code diff and executable code which resolves to an answer in the question (e.g. prints the loss). The code diff should be compact (<100 lines of code)

Aim for a fair, novel question: its outcome should be possible to reason about using the assignment, while testing an insight beyond the supplied examples. Course staff may use interesting questions in a future quiz.

We expect to use 0-2 quiz question per quiz from students. Bonus points for being selected *and for difficulty*

Logistics (Quizzes)

We have 7 quizzes, one for each assignment

- Quizzes happen roughly every 1.5 weeks (except the first one, in 1 week).
- Each quiz is a series of “prediction problems”
 - Every question comes with a diff that *fully specifies a run and metrics*
 - There is a objective answer to each question that can be mechanically graded
- Partial credit
 - You can “cite” your assignment in your quiz rationale. Citations to valid assignment experiments get up to 75% of the points back.
 - You *cannot* bring in your assignments to the quiz. You need to know what you ran.

AI use disclosure: we will use LLMs for mechanical aspects of grading (e.g. matching quiz rationales to assignments)

Problem difficulty

Example Quiz Question (Difficulty Easy): We train each of the following depth-8 transformer variants using the same training recipe and hyperparameters. Rank the variants from lowest to highest final validation loss. Use $<$, $>$, or $=$ between the letters; use $=$ when the losses are within 0.01.

- A. Baseline
- B. No prenorm
- C. No residual
- D. No MLP
- E. No attention
- F. No positional embeddings

► No prenorm diff

► No residual diff

► No MLP diff

► No attention diff

► No positional embeddings diff

Example Quiz Question (Difficulty Hard): We train the default 8 layer transformer as well as a variant where instead of having the residual connection come from the previous layer, we take 0.5 contribution from the previous layer and 0.5 contribution from two layers before (except for the first layer). For each method, we take the best loss over six learning rates $\{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$. What is your prediction of the loss difference between each method ($L_{\text{mixed residual}} - L_{\text{default}}$)?

► Core diff

Quiz questions have varying difficulty

- **Easy:** If you did the assignment, and memorized the runs, you would get these
- **Medium:** If you follow guided, open-ended extrapolations, you would get these
- **Hard:** If you follow guided extrapolations + have intuition, you would get partial credit

Logistics (Quizzes)

How can I prepare for the quiz / how does the partial credit work?

- Run a broad range of experiments incl basic ones (so you can cite them)
- Understand and digest the experiments so you know what you're citing
- Come to tutorials prepared (do assignment, think about quiz solution)

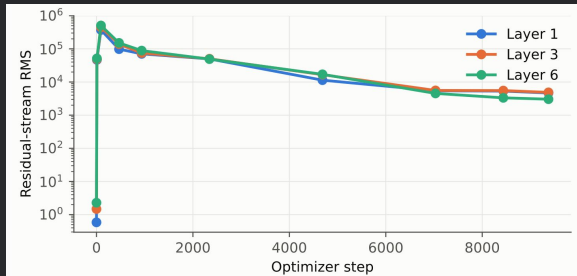
7x 40 minute Quizzes is alot!

- Yes, but this also means each question is lower stakes and gets averaged out
- Also, assignments are already 'quiz prep' - no extra time expected or required

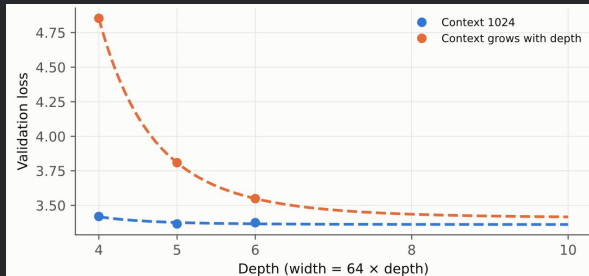
Logistics (Quizzes)

Isn't it unfair to predict arbitrary experiments? Why do I care if the loss is 3.1234...

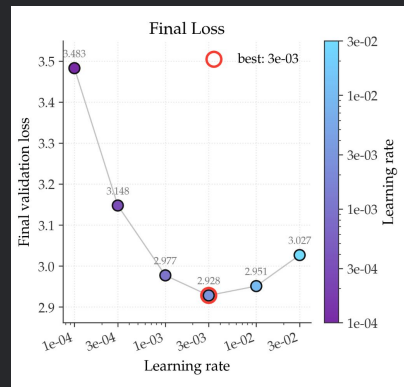
- We will pick outcome metrics that are relevant
- We view the whole class doing badly as our failure
- But also, the real world is unfair: your experiments depend on your prediction



Is this a stable run?



Will orange beat blue?



Is my baseline reasonable?

Note: don't overfit to our setting! We want to teach you a process and mindset

Logistics (Tutorials)

Tutorials are 30 minute, in-person discussions.

- 4 students, 1 instructor/CA
- Starts week 3 (past add/drop)
 - We will plan scheduling/groups in week 2-3
- Discuss previous assignment (and quiz)
- 15% of grade. Graded loosely on fail/low pass/high pass.
 - Fail: clearly did not do the assignment (or just ran claude code)
 - Low pass: understands the basics of what was run and why
 - High pass: demonstrates some level of generalization on assignment concepts
- If everyone does the assignment, this is just fun small group ML chats.

Logistics (Compute)

Compute generously sponsored by  **Modal**

Send us your stanford email - we will set you up for the class

We will send out slack, modal, code, and first assignment by tonight

Use common sense, DO NOT USE CLASS CREDITS FOR RESEARCH

- We have monitoring instrumentation
- We will refer you to OCS for honor code violations

Logistics (waitlist)

We are aware that demand for this class is high and we're maxed on waitlist.

If you want to take this course..

- If you're a graduating senior, fill out the google form
- Stay in the course, take the quizzes and do assignments (ask us for compute)
- If you can't join the waitlist, we will have a waitlistwaitlist.

Sorry for the capacity issues! We are doing our best.

Who is this class for?

Pros:

- Learn 'deep learning intuition'
- Learn via hands-on experiments
- Hear about cutting edge deep learning empirics
- Small group tutorials with expert instruction staff
- Feeling of mastery in a very specific small LM setting

Cons:

- This is a new course (and LM centric)!
- This is a very experimental course!
- There are alot of long exams!
 - Some questions will feel unfair
- You will learn about a very niche, small model.
- This is not a basic course
 - (I will not teach you backprop)
- We may lower your GPA

We think this is a fun experiment in a new way to teach 'alchemy'
If you also think this might be fun, this is a good course for you.

Setting of this class

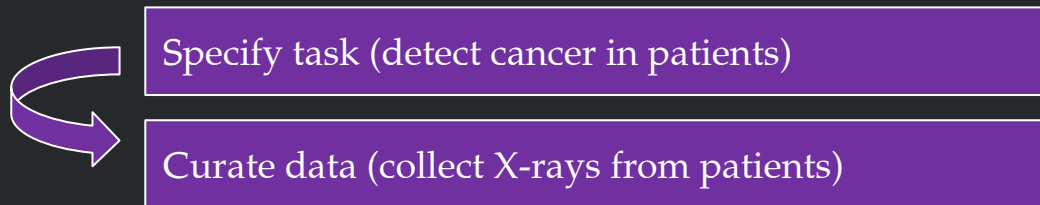
(what aspects of deep learning are we studying?)

Deep learning pipeline in practice

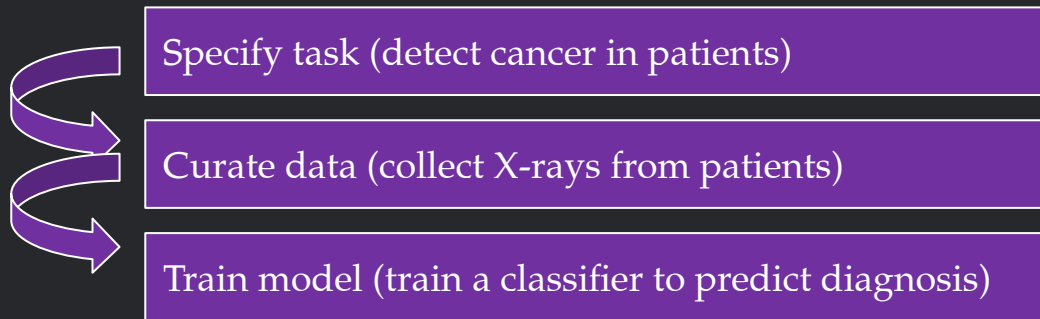
Deep learning pipeline in practice

Specify task (detect cancer in patients)

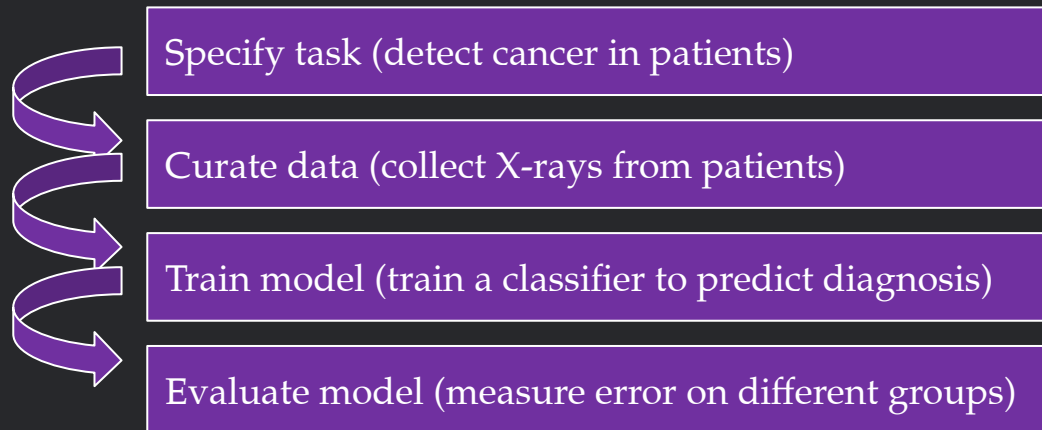
Deep learning pipeline in practice



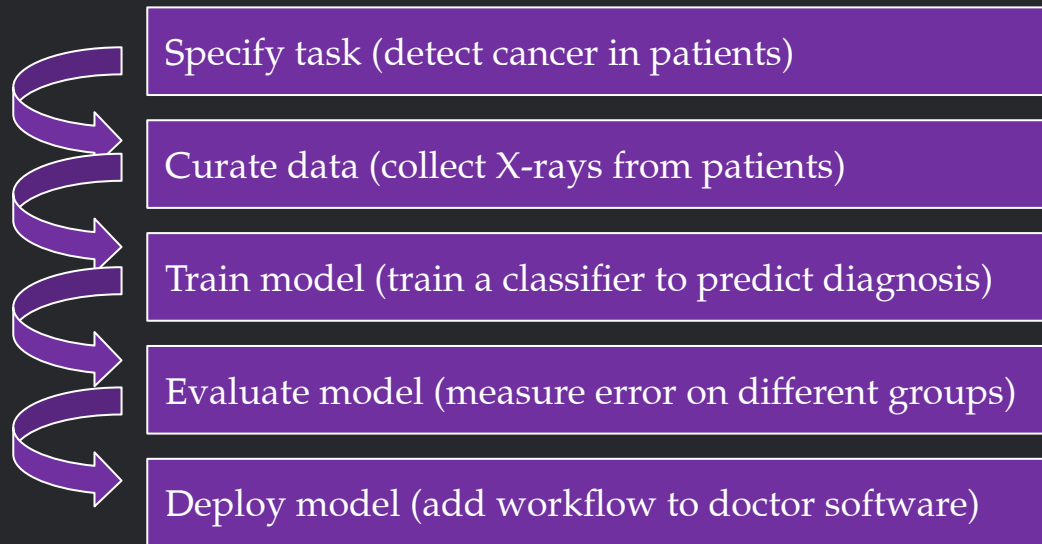
Deep learning pipeline in practice



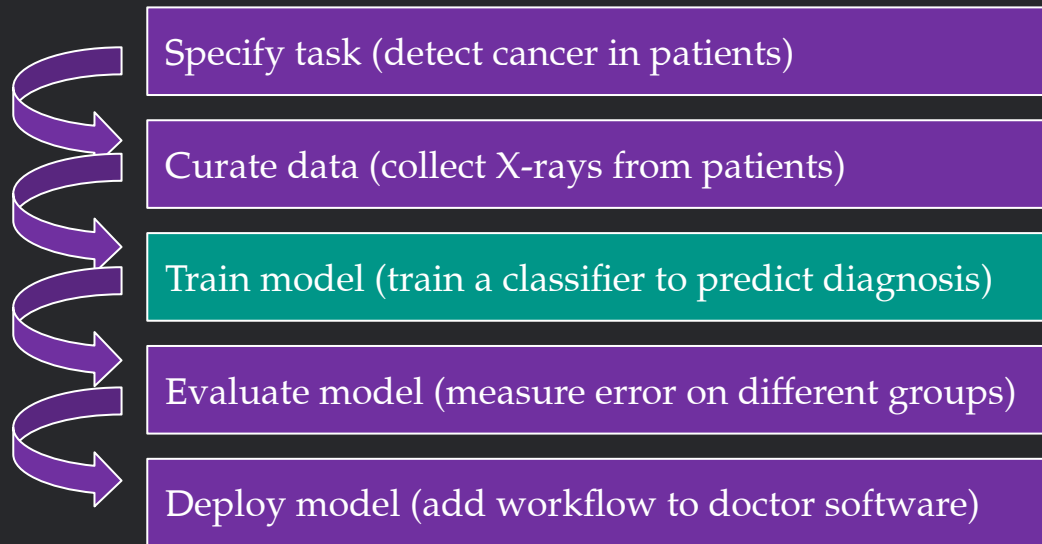
Deep learning pipeline in practice



Deep learning pipeline in practice



This class focuses on understanding training



Not necessarily the
most important

Instructors find it pretty
interesting and fun

Different types of training

Pre-training

Fine-tuning

Reinforcement learning

Alignment

Contrastive learning

RLHF

Most of this class focuses on pre-training

Goal of pre-training

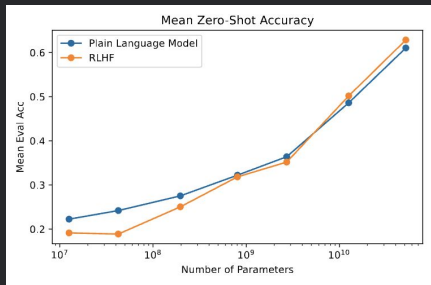
The goal of pre-training is to imitate a given data distribution (e.g. the internet).

Goal of pre-training

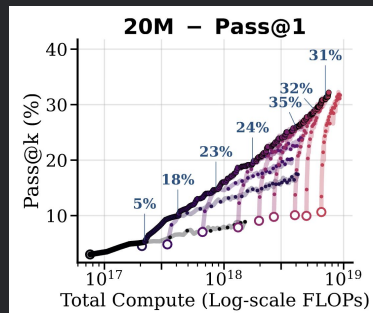
The goal of pre-training is to imitate a given data distribution (e.g. the internet). The last few years have found that minimizing pre-training loss leads to models that can be rapidly adapted to downstream tasks

1	thanks => merci	In-context learning ↓
2	hello => bonjour	
3	mint => menthe	
4	wall => mur	
5	otter => loutre	
6	bread => pain	

*GPT-3 level
pre-training unlocked
in-context learning*



*Anthropic's stronger
base models benefited
more from alignment*



*Better pre-trained chess
models led to
exponentially faster RL*

?

?

?

?

Why does this class study pre-training

1. Validation loss is predictive of downstream utility with relatively little noise
2. Pre-training can be studied at small scale due to scaling laws
(runs at small scale can predict the results of large scale experiments)
3. Pre-training is easier to reason about since all training starts from scratch

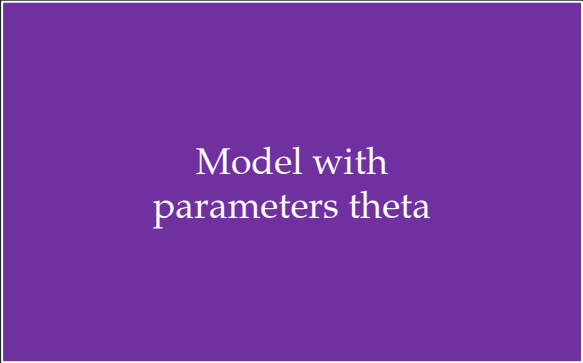
Pre-training language models

Since pre-training is most established for language models, this will be the settings for the first four assignments following standard practices

It is okay if you don't recognize all the terms we're going to discuss! On Wednesday 9/23, 5-7 PM, we will be hosting a supplementary lecture introducing basic concepts in deep learning and language modeling, following the LLMs 101 handout on the website (planning on being recorded). Will be direct/dry compared to the rest of this class

Cartoon of pre-training

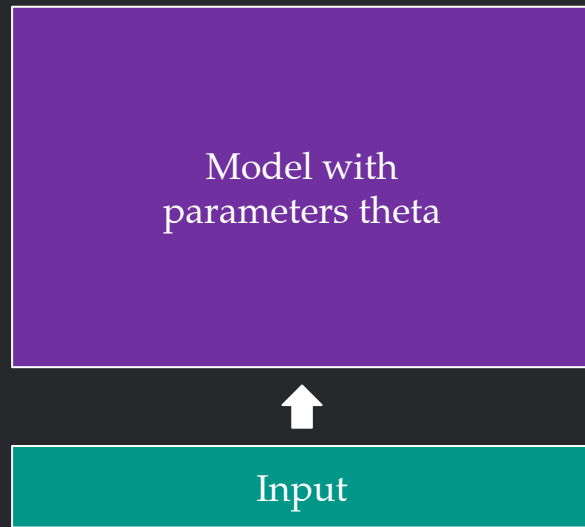
Four main parts when specifying a deep learning algorithm



Model with
parameters θ

Cartoon of pre-training

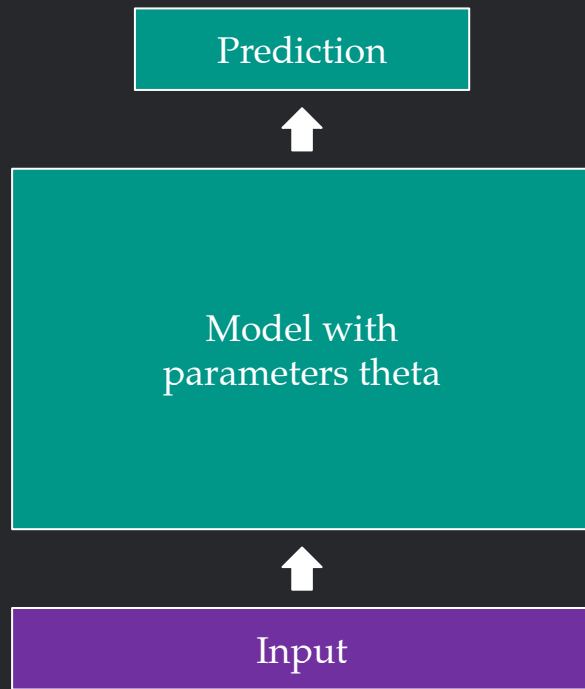
Data specifies a distribution over inputs to the model
(e.g. internet text)



Cartoon of pre-training

Data specifies a distribution over inputs to the model
(e.g. internet text)

Architecture specifies how θ maps to a model
prediction (e.g. transformer produces log likelihoods)

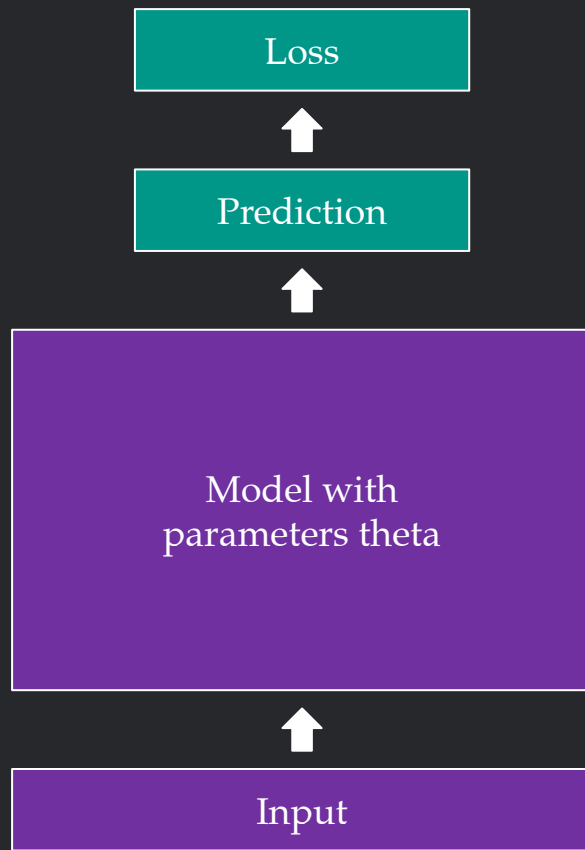


Cartoon of pre-training

Data specifies a distribution over inputs to the model
(e.g. internet text)

Architecture specifies how θ maps to a model
prediction (e.g. transformer produces log likelihoods)

Objective specifies error of θ via the loss function
(e.g. cross-entropy loss on predicted likelihoods)



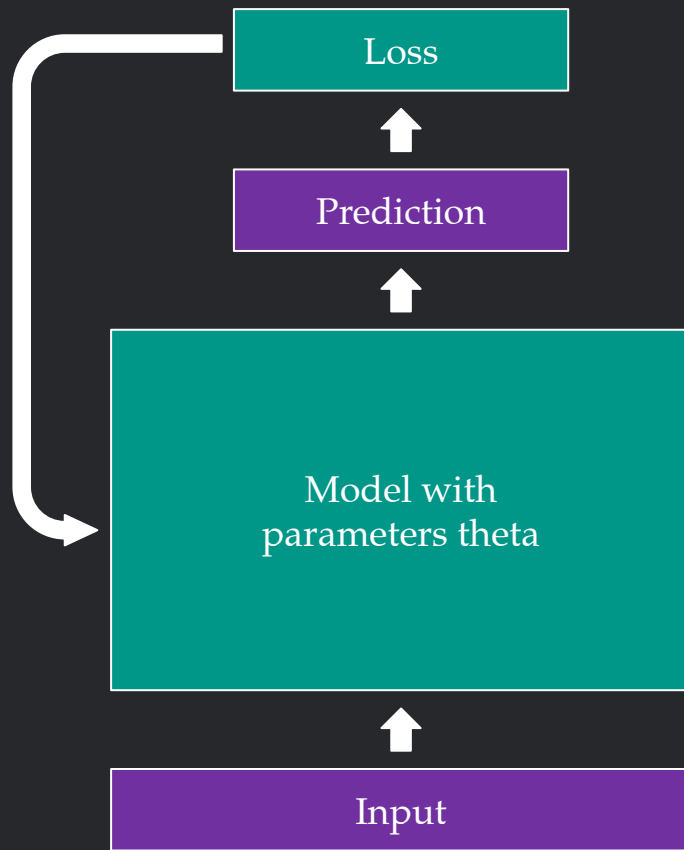
Cartoon of pre-training

Data specifies a distribution over inputs to the model (e.g. internet text)

Architecture specifies how θ maps to a model prediction (e.g. transformer produces log likelihoods)

Objective specifies error of θ via the loss function (e.g. cross-entropy loss on predicted likelihoods)

Optimization specifies how the model should update itself based on its loss (e.g. gradient descent on cross-entropy loss)



Experiments in this class

We have developed a simple small scale testbed with a default training recipe that you will all become very familiar with

Reflects many aspects of large scale deep learning while differing in others

The default run has ~35M parameters and 600M tokens, takes ~10 minutes on 1 H100 GPU, costs \$0.50 on Modal

Hyperparameter	Default	Notes
Optimizer		
learning_rate	0.003	peak learning rate
optimizer_name	adamw	adamw or sgd
beta1	0.9	Adam β_1 (momentum)
beta2	0.95	Adam β_2
lr_schedule	linear	linear, cos, constant, $\text{wsd} < \frac{\text{wsd}}{\text{total_steps}}$ (e.g. $\text{wsd} 0.2$: stable then linear decay over the last 20%)
warmup_percent	0.01	linear warmup, as a fraction of total steps
weight_decay	0.1	masked: no decay on embeddings, norms, or biases
grad_norm	1.0	gradient-clipping threshold; set to None to disable
Data		
num_train_sequences	600,000	training horizon; each sequence is 1024 tokens
num_epochs	1.0	fractional values allowed, > 1 repeats data
batch_size	64	sequences per optimizer step
context_length	1024	fixed by the preprocessed dataset
data_seed	42	shuffle order of training sequences
Model		
depth (d4-d18)	d8	built-in ladder: layers = depth, hidden = 64·depth, heads = hidden/64, MLP = 3.5× hidden, head_dim=64, vocab_size=4096, context length 1024. Custom LMConfigs can override these choices.
qk_norm	true	normalize queries and keys before attention dot products
tie_word_embeddings	false	share input and output embedding weights when true
model_seed	42	initialization randomness
dropout	0.0	shared attention and residual-branch dropout probability
Systems		
num_micro_batches	1	gradient accumulation chunks per optimizer step; usually only needed above batch size 256
deterministic	false	request deterministic PyTorch behavior when possible
precision	mp	mp, bf16, or fp32

Default objective

Our model Q_θ predicts the likelihood of a sequence x from ground-truth distribution P . We will use auto-regressive models Q_θ , which represent

$$Q_\theta(x) = \prod_i Q_\theta(x_i \mid x_{<i})$$

We measure Q_θ 's error via cross-entropy loss $L(\theta)$, or the negative log likelihood assigned by Q_θ to real samples (often a held-out validation set)

$$L(\theta) = \mathbb{E}_{x \sim P}[-\log Q_\theta(x)] = \mathbb{E}_{x \sim P}[-\sum_i \log Q_\theta(x_i \mid x_{<i})]$$

We then normalize by the sequence length to measure loss per token. We want the loss to be as low as possible.

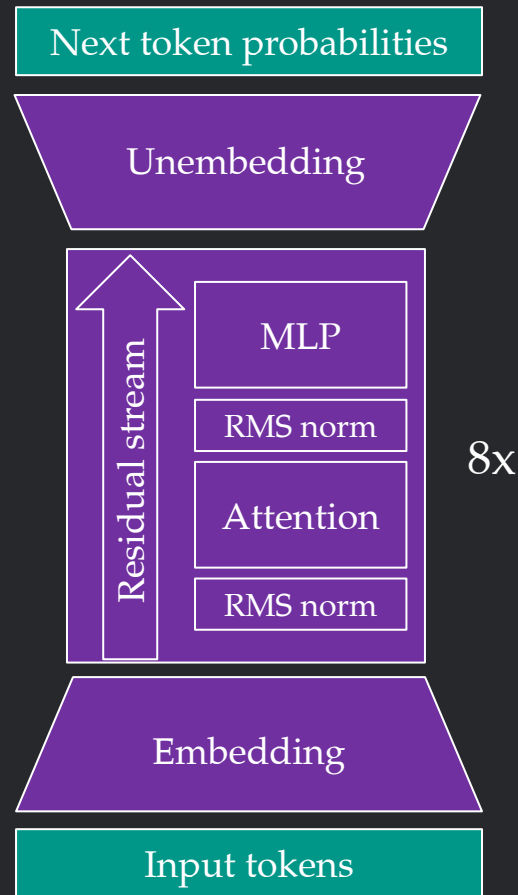
Default architecture

We will be using a Llama-style architecture with 8 layers

- Embedding layer
- 8 layers, each consisting of
 - Attention (with RoPE positional embeddings and QK norm)
 - MLP (with SwiGLU activation)
 - Residual connections and prenorm
- Unembedding layer with softmax

We typically train deeper models with a fixed aspect ratio

This architecture will process sequences of length 1024

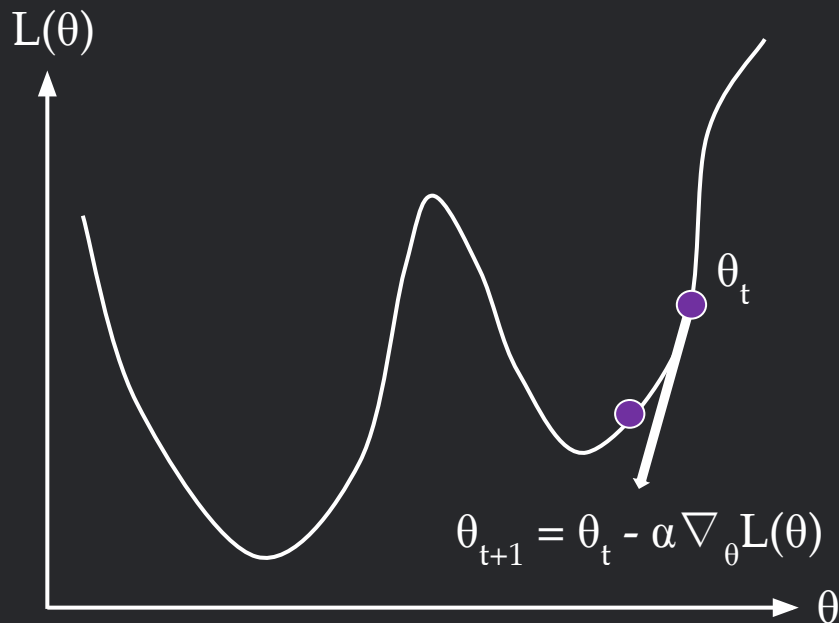


Default optimization

We optimize our models using stochastic gradient descent

Minibatch training with batch size 64 and the AdamW optimizer

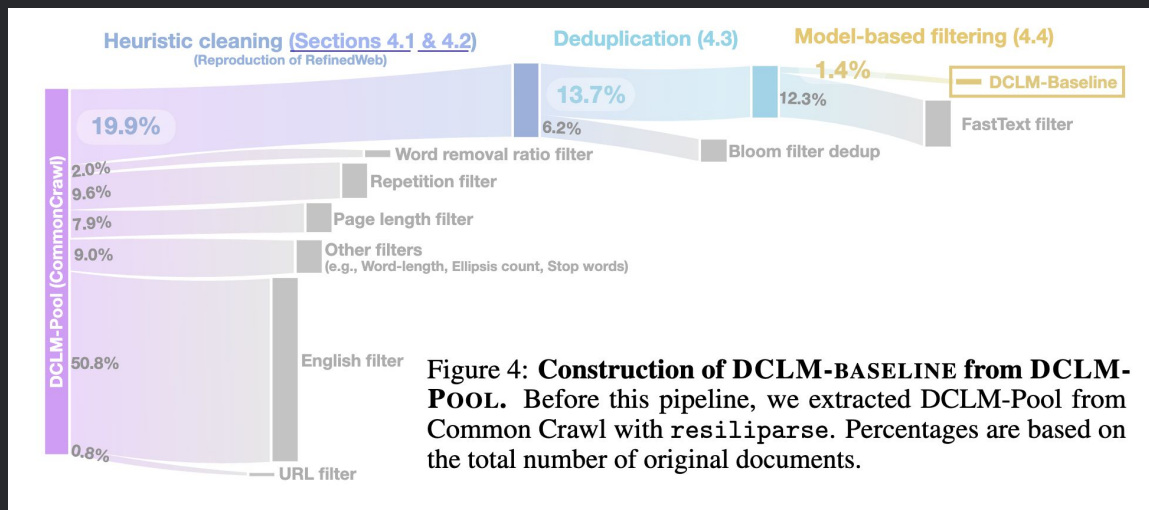
We schedule the learning rate α to quickly warm up and linearly decay for the rest. We improve stability via weight decay and gradient clipping



Disclaimer: do not trust schematic!
You will build your own intuition in Opt 1 + 2

Default data (fixed)

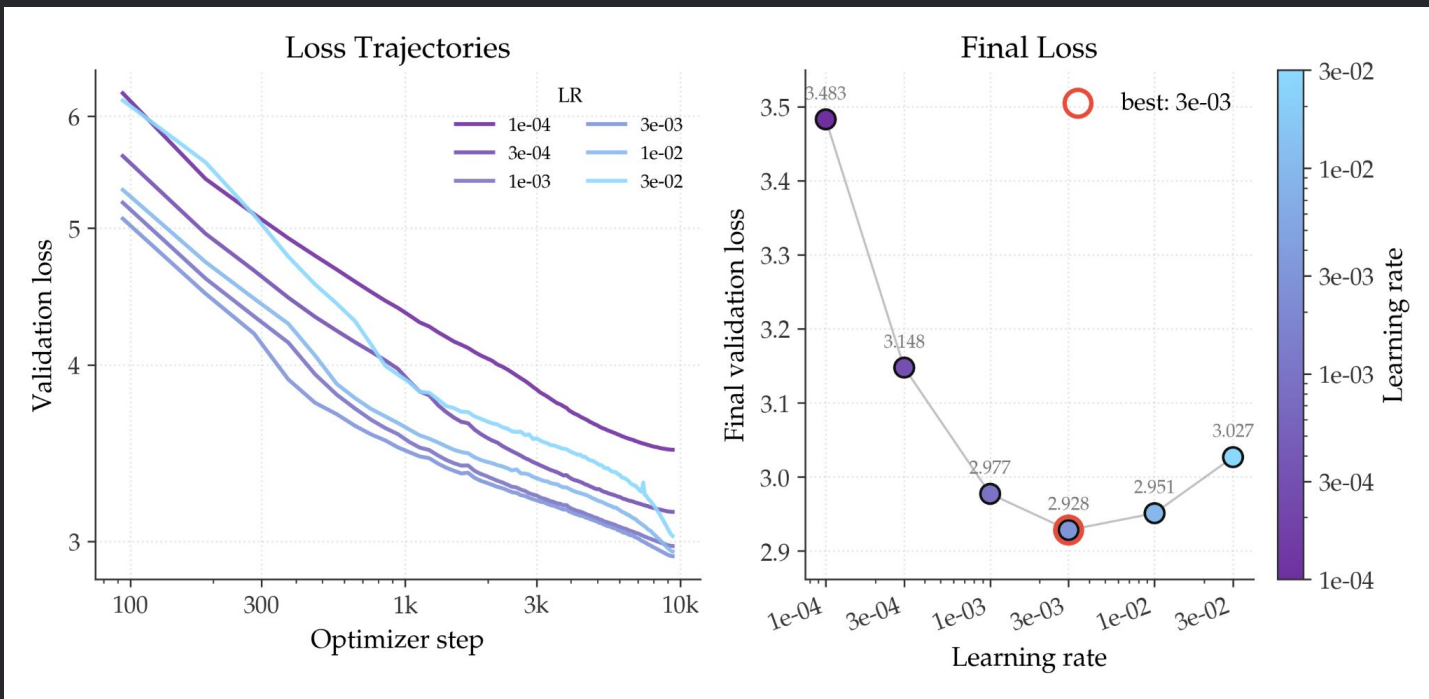
We will be using DCLM, data curated from internet web dumps. We will represent the text with 4096 discrete tokens. Following standard practice, we will take all the documents, concatenate them, and form 1024 token sequences representing natural data. Later in the class, we will switch to another data modality like DNA



Experiment walkthrough

Example runs

Here is a default model trained for six different learning rates



Example assignment question

At the start of the unit, you will get an assignment suggesting exploration. Some parts will be more structured, whereas others will be more exploratory

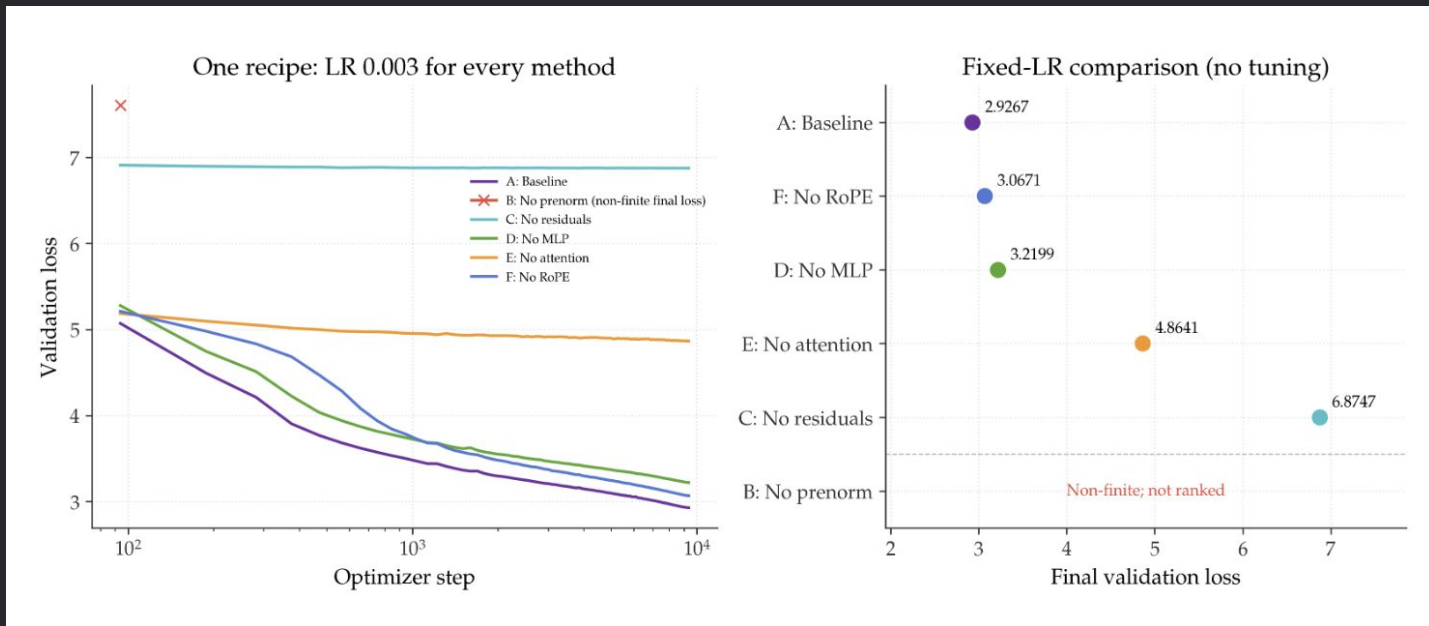
Problem 1: What does each transformer component contribute?

A transformer consists of many architectural components. In this assignment problem, we will determine the relative importance of each transformer component in its ability to both represent functions and optimize smoothly.

- (a) **Change one component at a time.** Start from the reference model and independently bypass the block prenorms, both residual connections in every block, the MLPs, attention, and rotary position encoding. Use the definitions on the following pages. Before running each comparison, write down a prediction and the information or computation that remains available to the modified model. Keep the data, training horizon, and optimization settings fixed. Plot the validation trajectories and report the final loss difference from the reference.
- (b) **Investigate the residual pathway.** Propose a different way to carry information across blocks, such as scaling a skip connection. Write its update rule and specify the treatment of the first block. Compare it with the reference using matched learning-rate searches.

Example exploration part (a)

Part (a) suggests I try training models while removing each component. Therefore, I launched 6 default runs while modifying each component and measure loss

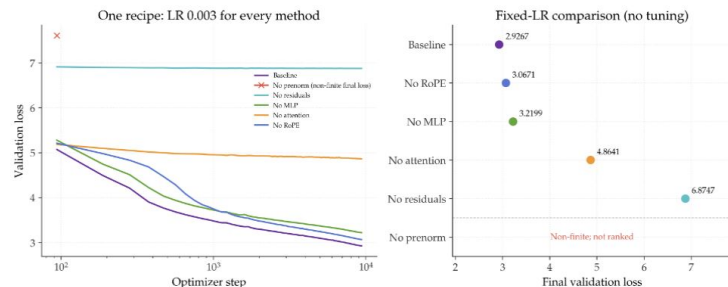


Example report part (a)

In my report, I might add these plots as well as an explanation of what I ran

Part (a): Start with the default learning rate

I trained the reference and each independent ablation using the same default recipe, including peak LR 0.003. This was six runs, not an LR sweep. The figure uses the full 600,000-sequence runs and their final evaluations; no alternative learning rates enter this comparison.



Observation. From lowest to highest finite final loss, the ordering is reference, no RoPE, no MLP, no attention, then no residuals. No prenorm becomes non-finite and is not assigned a finite rank. Its last finite evaluation is not substituted for the final loss; the cross on its trajectory shows a finite observation before numerical failure.

Interpretation. The components have different effects under one shared recipe. Removing attention or both residuals produces much worse loss than removing RoPE or the MLP. The no-prenorm run raises a separate question about numerical stability and optimization, not just prediction quality.

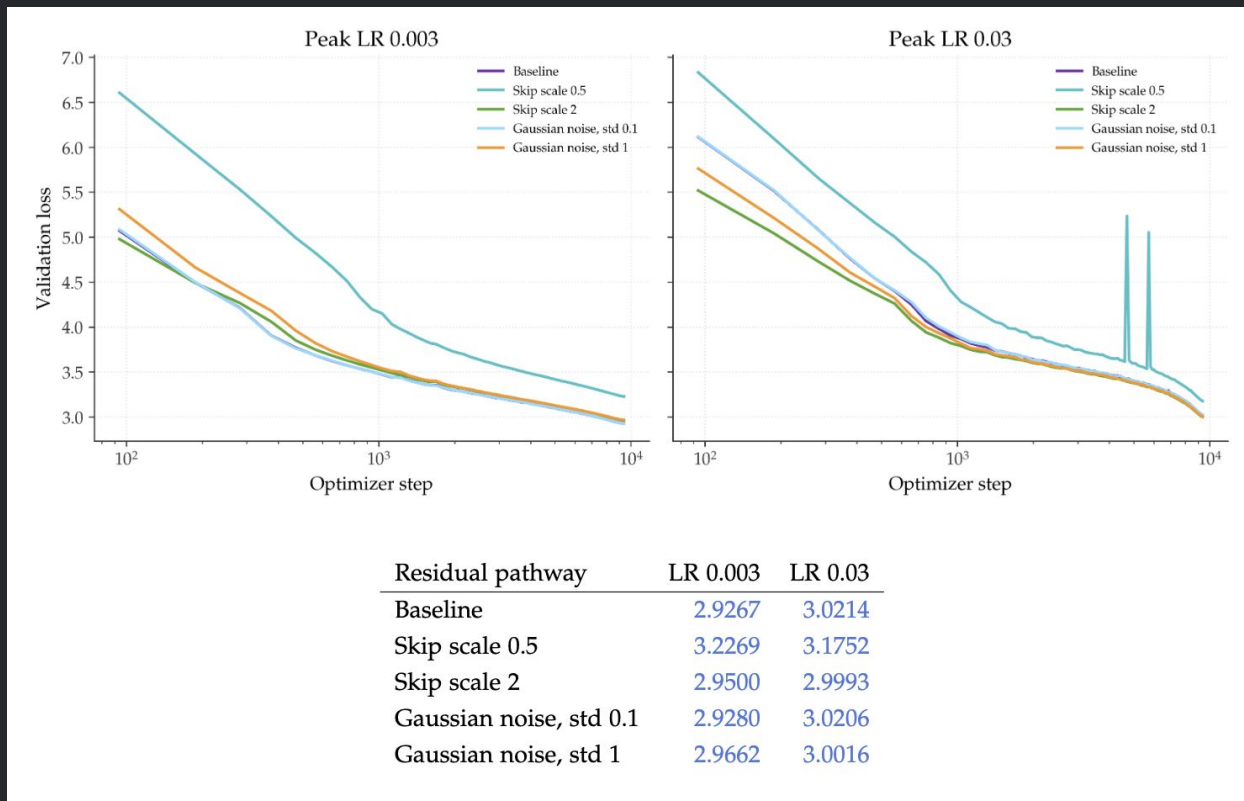
Limitations and next steps. I have only compared these methods at one LR. A learning rate that works for the reference need not work for an ablation. I would next test whether a smaller LR rescues no prenorm and whether the other methods prefer different LRs, giving each method the same search budget. Failure at one LR is not evidence of failure at all LRs. Repeating the runs with paired model/data seeds would help assess the reliability of smaller gaps.

Example exploration part (b)

We need to understand the role of the residual connection in training. I designed the following eight runs to improve my understanding

- Scaling each skip connection by a fixed constant (0.5 or 2.0)
- Adding random gaussian noise to each (std 0.1 or 1.0)
- Ran each experiment at two learning rates (0.003 (default) and 0.03)

Example exploration part (b)



Pop quiz?!

Example question 1 (easy)

Difficulty: Easy

Train A–F independently with otherwise default d8 settings at peak LRs

$$\mathcal{G} = \{3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}\}.$$

Rank their best *final* validation losses: $L_j^* = \min_{\eta \in \mathcal{G}} L_j(\eta)$. Order from lowest to highest using $<$.

Label	Variant	Change
A	Reference	Native d8, with all standard components.
B	No prenorm	Replace both block-input norms with identities.
C	No residuals	Apply normalized attention followed by normalized MLP, with no skip additions.
D	No MLP	Bypass each MLP branch.
E	No attention	Bypass each attention branch.
F	No RoPE	Use identity rotations for queries and keys.

Example question 1 (easy) diff

```
@@ Block.__init__ (B: no prenorm)
-self.input_layernorm = layer.input_layernorm
-self.post_attention_layernorm = layer.post_attention_layernorm
+self.input_layernorm = nn.Identity()
+self.post_attention_layernorm = nn.Identity()

@@ Block.forward (C: no residuals)
-h = x + self.attention_update(x, position_embeddings, attention_mask)
-return h + self.mlp_update(h)
+a = self.attention_update(x, position_embeddings, attention_mask)
+return self.mlp_update(a)

@@ Block.mlp_update (D: no MLP)
-m = self.mlp(self.post_attention_layernorm(x))
-return F.dropout(m, p=self.dropout, training=self.training)
+return torch.zeros_like(x)

@@ Block.attention_update (E: no attention)
-a = self.self_attn(self.input_layernorm(x),
-                  position_embeddings, attention_mask)
-return F.dropout(a, p=self.dropout, training=self.training)
+return torch.zeros_like(x)

@@ Backbone.forward (F: no RoPE)
  position_embeddings = self.rotary_emb(x, position_ids)
+cos, sin = position_embeddings
+position_embeddings = (torch.ones_like(cos), torch.zeros_like(sin))
```

Example question 2 (hard)

Difficulty: Hard

Compare default d8 with a mixed-skip variant. Both compute

$$a_\ell = A_\ell(x_\ell), \quad m_\ell = M_\ell(x_\ell + a_\ell).$$

Here x_ℓ is block ℓ 's input; A_ℓ, M_ℓ include prenorm.

$$\text{Reference: } x_{\ell+1} = x_\ell + a_\ell + m_\ell,$$

$$\text{Modified: } x_{\ell+1} = \frac{1}{2}x_\ell + \frac{1}{2}x_{\ell-1} + a_\ell + m_\ell.$$

The first block stays unchanged. Gradients flow through both skip inputs.

Train both architectures with otherwise default settings at peak LR

$$\mathcal{G} = \{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}\}.$$

Using *final* validation loss, predict

$$\Delta L = \min_{\eta \in \mathcal{G}} L_{\text{mixed}}(\eta) - \min_{\eta \in \mathcal{G}} L_{\text{reference}}(\eta).$$

A negative value favors the mixed variant.

```
@@ Backbone.forward
+x_prev = None
  for block in self.layers:
-   x = block(x, position_embeddings, attention_mask)
+   block_input = x
+   if x_prev is None:
+       x = block(x, position_embeddings, attention_mask)
+   else:
+       a, m = block.updates(x, position_embeddings, attention_mask)
+       residual = 0.5 * x + 0.5 * x_prev
+       x = (residual + a) + m
+   x_prev = block_input
```

Example questions

Now, take the quiz! (while providing some demographic information)

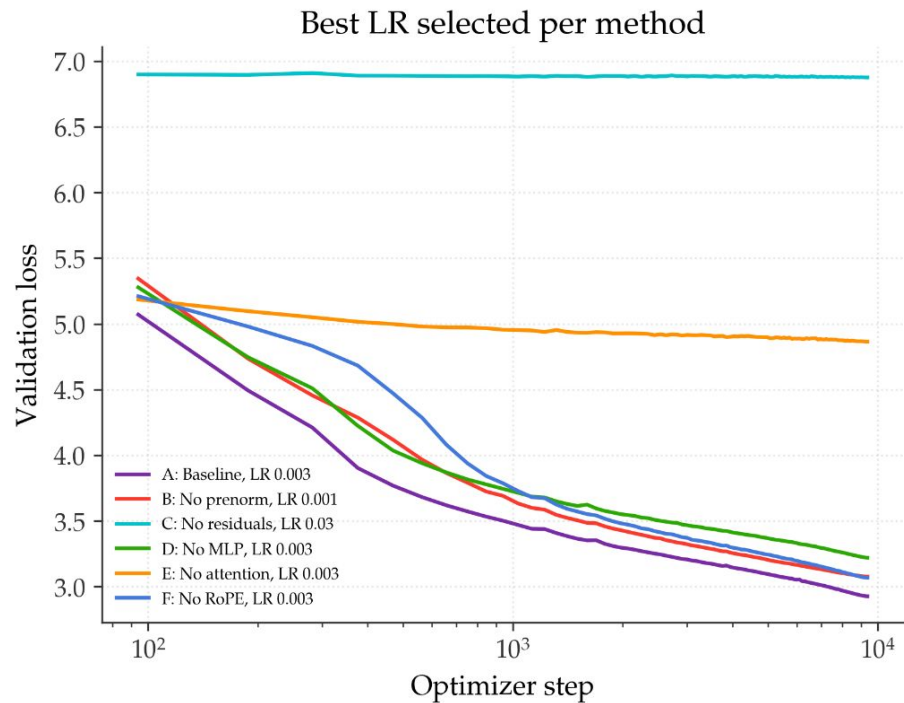
The quiz questions are available on the form

bit.ly/312quiz0



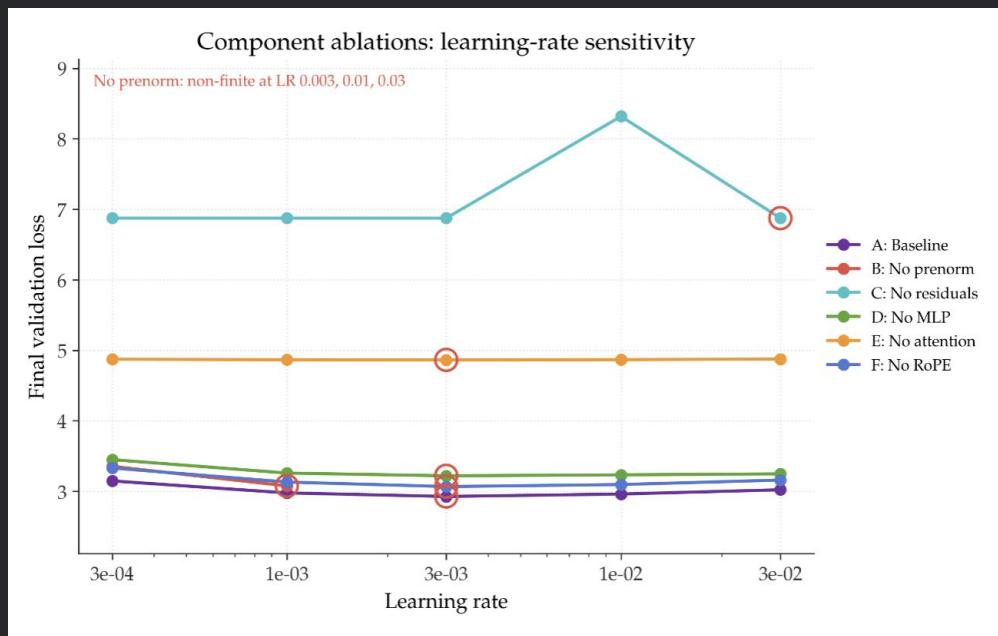
Answer to question 1

Variant	Intervention	Best LR	Final loss	Gap to A
A	Baseline	0.003	2.9267	+0.0000
B	No prenorm	0.001	3.0766	+0.1498
C	No residuals	0.03	6.8745	+3.9478
D	No MLP	0.003	3.2199	+0.2931
E	No attention	0.003	4.8641	+1.9373
F	No RoPE	0.003	3.0671	+0.1403



Commentary on question 1

Though our experiments in the assignment gave us some insight, it missed the fact that prenorm doesn't train at default/higher learning rates



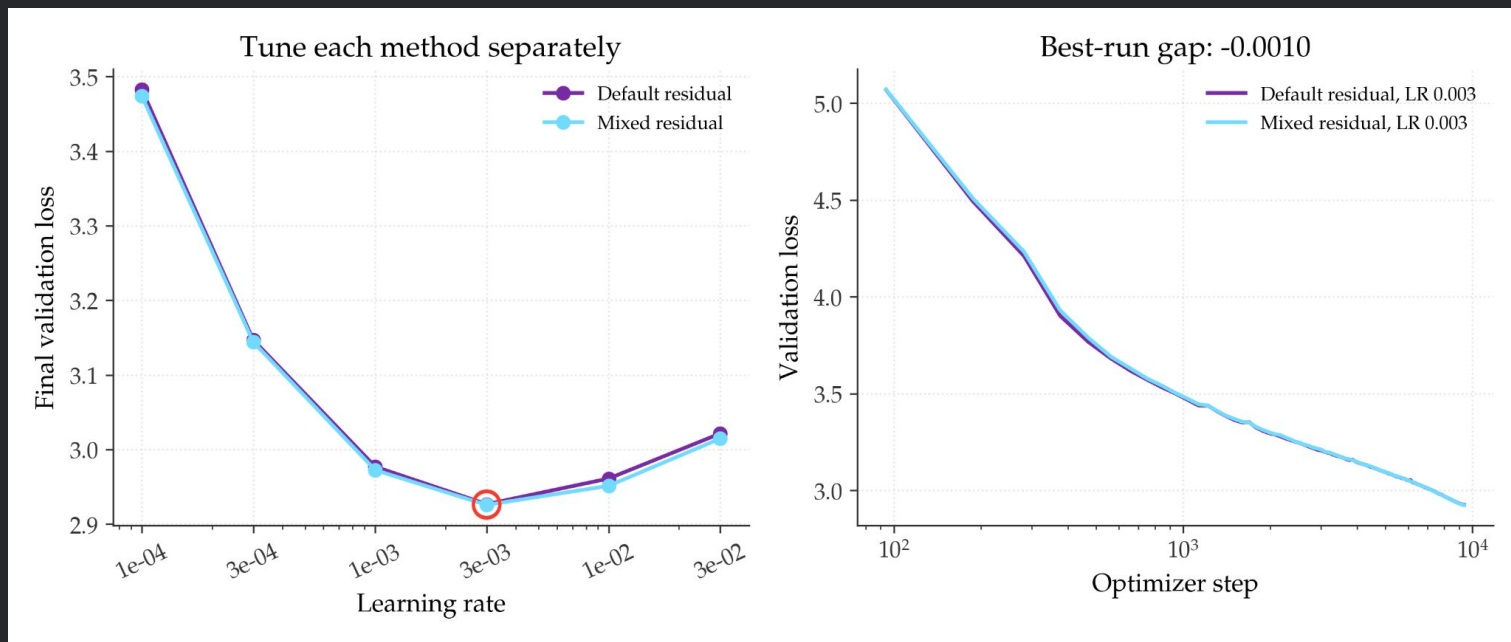
Answer to question 2

$$\Delta L = 2.9257 - 2.9267 = -0.0010.$$

Peak LR	Default residual	Mixed residual
0.0001	3.4828	3.4740
0.0003	3.1470	3.1443
0.001	2.9770	2.9724
0.003	2.9267	2.9257
0.01	2.9611	2.9514
0.03	3.0214	3.0145

Commentary on question 2

In line with some of our sample experiments, the residuals are relatively robust to major modifications



Thanks for coming!

We're excited to have you in our class!

Grab a Hi-chew on the way out!



If you are not able to enroll on the waitlist for technical reasons, please utilize the following form to enter our waitlist for the waitlist. Practically, this will only matter for graduating students

bit.ly/312wlwl